



Name: SHUBHAM

UID: 24BAI70452

Branch: AIT-CSE (AIML)

Section/Group: 24AIT_KRG-1A

Semester: 5

Subject Code: 24CSP-310

Subject Name : Advanced Database Management System

Experiment-03

Aim:

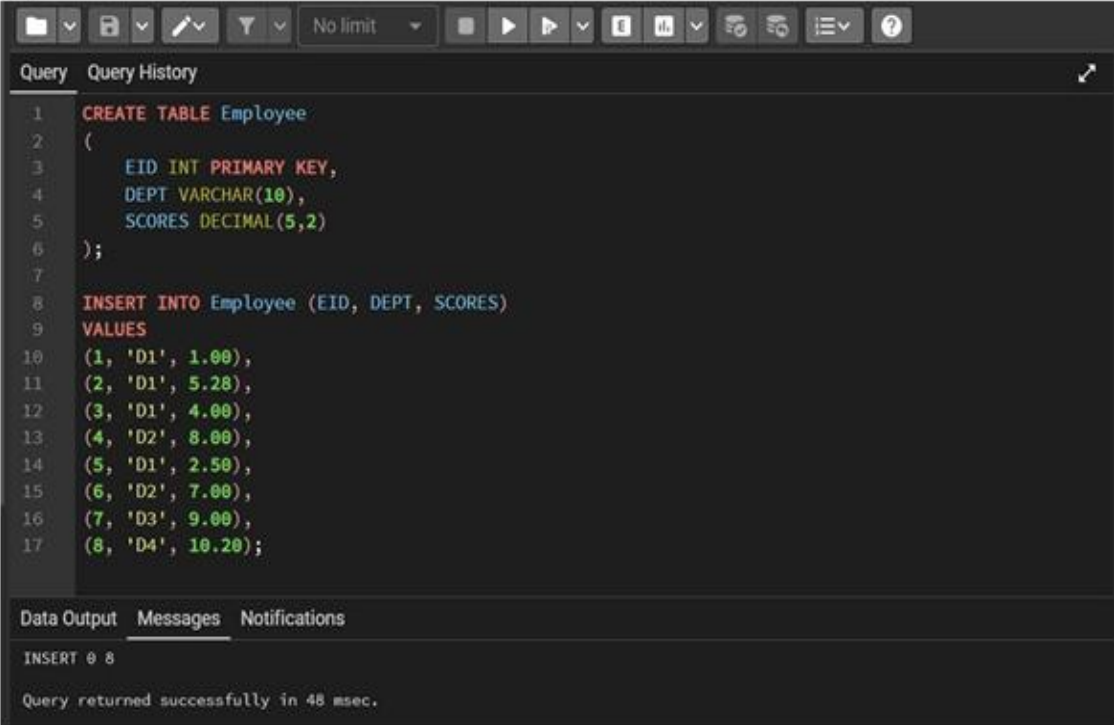
- A. You are given with the Employee schema; the task is to find the highest score for each respective department.
For eg: department D1 have the highest score as 5.28. Hence in the output, final updated score for D1 should be 5.28. Same with the other departments i.e., D2, D3 and D4.
- B. A company maintains records of products sold by different salespersons. The management wants to prepare a sales performance report by identifying the salesperson(s) who generated the highest overall sales revenue. If multiple salespersons have the same highest total sales amount, all of them must be included in the final report. Return the seller ID(s) satisfying this condition.
- C. A company maintains employee salary records for different departments. The management wants to prepare a report containing employees whose salaries fall among the top three highest distinct salary values within their respective departments. If multiple employees receive the same qualifying salary, all such employees must be included in the final report. Display the department name, employee name, and salary for every qualifying employee.

Tools / Platform Used:

- Windows
- PostgreSQL
- PgAdmin

Code and Output:

A.



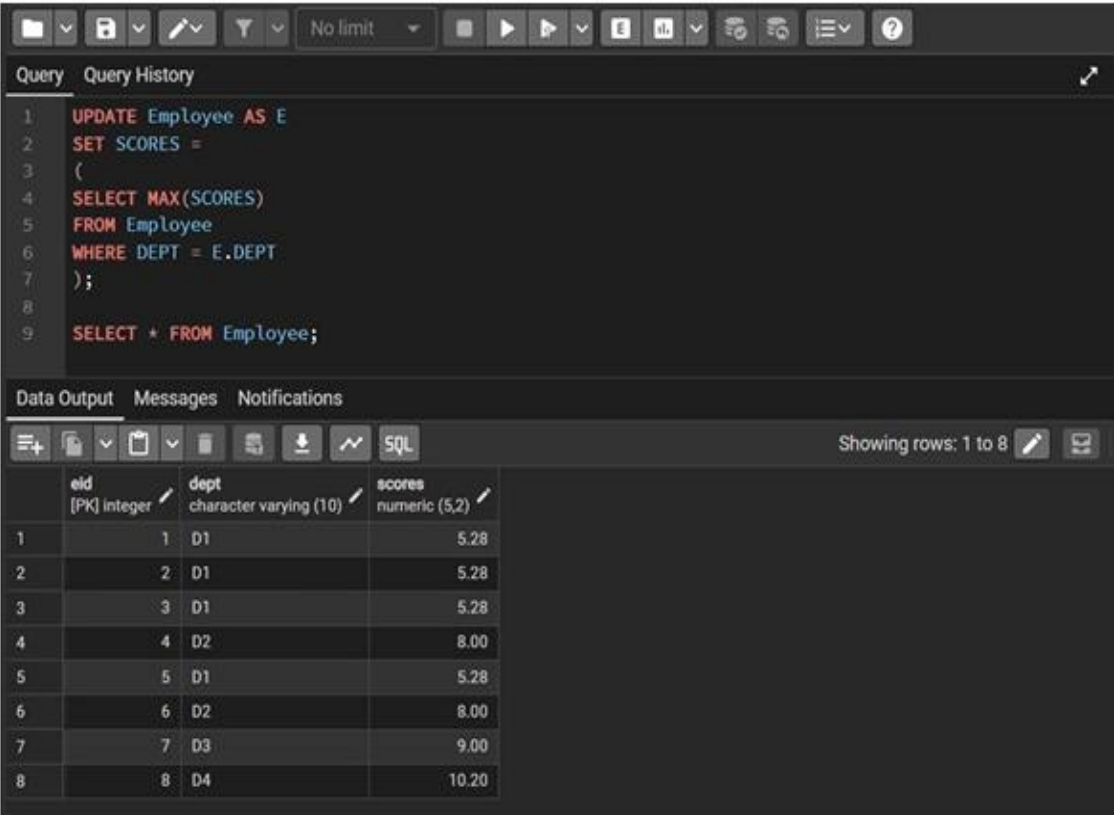
The screenshot shows a SQL IDE interface with a dark theme. The top toolbar includes icons for file operations, query execution, and settings. The main editor displays SQL code for creating a table and inserting data. The bottom panel shows the 'Messages' tab with a confirmation message.

```
1 CREATE TABLE Employee
2 (
3     EID INT PRIMARY KEY,
4     DEPT VARCHAR(10),
5     SCORES DECIMAL(5,2)
6 );
7
8 INSERT INTO Employee (EID, DEPT, SCORES)
9 VALUES
10 (1, 'D1', 1.00),
11 (2, 'D1', 5.28),
12 (3, 'D1', 4.00),
13 (4, 'D2', 8.00),
14 (5, 'D1', 2.50),
15 (6, 'D2', 7.00),
16 (7, 'D3', 9.00),
17 (8, 'D4', 10.20);
```

Data Output Messages Notifications

INSERT 0 8

Query returned successfully in 48 msec.



The screenshot shows the same SQL IDE interface. The main editor displays an SQL update query. The bottom panel shows the 'Data Output' tab with a table of 8 rows. The table has columns for eid, dept, and scores. The 'Showing rows: 1 to 8' text is visible on the right.

```
1 UPDATE Employee AS E
2 SET SCORES =
3 (
4     SELECT MAX(SCORES)
5     FROM Employee
6     WHERE DEPT = E.DEPT
7 );
8
9 SELECT * FROM Employee;
```

Data Output Messages Notifications

Showing rows: 1 to 8

	eid [PK] integer	dept character varying (10)	scores numeric (5,2)
1	1	D1	5.28
2	2	D1	5.28
3	3	D1	5.28
4	4	D2	8.00
5	5	D1	5.28
6	6	D2	8.00
7	7	D3	9.00
8	8	D4	10.20

B.

The screenshot shows a SQL IDE interface with a query editor and a results pane. The query is as follows:

```
1 SELECT seller_id
2 FROM Orders
3 GROUP BY seller_id
4 HAVING SUM(Amount) = (
5     SELECT MAX(Max_Amt)
6     FROM (
7         SELECT seller_id, SUM(AMOUNT) AS Max_Amt
8         FROM Orders
9         GROUP BY seller_id
10    )
11 );
```

The results pane shows the following output:

seller_id	integer
1	3

C.

The screenshot shows a SQL IDE interface with a query editor and a messages pane. The query is as follows:

```
1 CREATE TABLE Department (
2     id INT PRIMARY KEY,
3     name VARCHAR(50)
4 );
5
6 CREATE TABLE Employee (
7     id INT PRIMARY KEY,
8     name VARCHAR(50),
9     salary INT,
10    departmentId INT,
11    FOREIGN KEY (departmentId)
12    REFERENCES Department(id)
13 );
14
```

The messages pane shows the following output:

```
CREATE TABLE
Query returned successfully in 49 msec.
```

The screenshot shows a SQL IDE with a toolbar at the top. The 'Query' tab is active, displaying the following SQL code:

```

1  INSERT INTO Department (id, name)
2  VALUES
3  (1, 'IT'),
4  (2, 'Sales');
5
6  INSERT INTO Employee (id, name, salary, departmentId)
7  VALUES
8  (1, 'Joe', 85000, 1),
9  (2, 'Henry', 80000, 2),
10 (3, 'Sam', 60000, 2),
11 (4, 'Max', 90000, 1),
12 (5, 'Janet', 69000, 1),
13 (6, 'Randy', 85000, 1),
14 (7, 'Will', 70000, 1);

```

Below the query editor, the 'Data Output' tab is selected, showing the message: "INSERT 0 7" and "Query returned successfully in 60 msec."

The screenshot shows the same SQL IDE with a new query in the 'Query' tab:

```

1  SELECT
2      D.NAME AS DEPARTMENT,
3      E.NAME AS EMPLOYEE,
4      E.SALARY AS SALARY
5  FROM
6      EMPLOYEE AS E
7  JOIN
8      DEPARTMENT AS D
9  ON
10     E.departmentId = D.ID
11 WHERE (
12     SELECT COUNT(DISTINCT E2.SALARY)
13     FROM EMPLOYEE AS E2
14     WHERE E2.departmentId = E.departmentId
15     AND
16     E2.SALARY > E.SALARY
17 ) < 3
18 ORDER BY D.NAME;

```

The 'Data Output' tab is selected, showing the results of the query. The status bar indicates "Showing rows: 1 to 6".

	department character varying (50)	employee character varying (50)	salary integer
1	IT	Joe	85000
2	IT	Max	90000
3	IT	Randy	85000
4	IT	Will	70000
5	Sales	Henry	80000
6	Sales	Sam	60000

Result:

The queries for the given tasks were written and executed successfully. The tasks given were completed successfully.

Learning Outcomes:

- Practiced PostgreSQL
- Got hands-on experience on working with subqueries in PostgreSQL.
- Practiced commands like GROUP BY and ORDER BY in PostgreSQL